# Edge ML Computer Vision Project Assignment

**Course: IESTI05 - Edge AI Engineering Part 1 Final Project: Fixed Function AI Implementation**

## Project Overview

This project serves as the culmination of Part 1 of the course, where you will apply all the concepts learned in Weeks 1-7 to create a practical Edge ML computer vision system. Working in groups of 3 students, you will design, implement, and deploy a complete computer vision solution on the Raspberry Pi Zero 2W.

This project represents a significant milestone in your Edge AI journey, combining theoretical knowledge with practical implementation skills. Success requires careful planning, systematic execution, and effective teamwork. The experience will prepare you for real-world edge AI development challenges and provide a strong foundation for Part 2 of the course.

> Good luck with your projects, and remember that the learning process is as valuable as the final results!

## Project Requirements

### Core Technical Requirements

1. **Hardware Platform**: Raspberry Pi Zero 2W with camera module
2. **AI Model**: Must use either image classification or object detection (or both)
3. **Dataset**: Use public datasets and/or create your own annotated dataset
4. **Deployment**: Final system must run in real-time on the Pi Zero 2W
5. **Performance**: Achieve reasonable inference speed (≥0.5 FPS) and accuracy (≥70%)

### Software Requirements

- Python-based implementation (scripts)
- Use TensorFlow Lite or Ultralytics for model inference.
- PIL for image processing
- Flask for local web deployment (if needed)
- Proper error handling and logging
- Clean, well-commented code

## Project Categories (Suggestions only)

### Category A: Agricultural/Environmental Monitoring

**Plant Disease Detection**

- Create a system that identifies diseases in plant leaves
- Use public datasets like PlantVillage or collect your own samples
- Classify healthy vs. diseased leaves and identify specific diseases
- *Extension*: Include severity assessment or treatment recommendations

**Fruit/Vegetable Quality Assessment**

- Build a system to grade fruits or vegetables by quality
- Classify items as fresh, ripe, or overripe
- Use public datasets or create your own with local produce
- *Extension*: Estimate shelf life or market value

**Pest Detection in Agriculture**

- Identify common agricultural pests in crop images
- Use existing insect datasets or create specialized collections
- Distinguish between beneficial and harmful insects
- *Extension*: Integrate with automated alert systems

## Category B: Industrial/Safety Applications

**Safety Equipment Compliance**

- Detect proper use of safety equipment (helmets, masks, gloves)
- Use industrial safety datasets or create workplace scenarios
- Implement real-time monitoring with alert systems
- *Extension*: Track compliance statistics over time

**Product Quality Control**

- Identify defects in manufactured products
- Create datasets of good vs. defective items
- Implement automated pass/fail classification
- *Extension*: Categorize defect types and severity

**Equipment Monitoring**

- Monitor equipment status through visual indicators
- Detect proper operation, maintenance needs, or failures
- Use industrial equipment datasets or create your own
- *Extension*: Predict maintenance schedules

## Category C: Smart Environment/IoT

**Parking Space Detection**

- Identify occupied vs. empty parking spaces
- Use aerial or ground-level parking datasets
- Implement real-time occupancy monitoring
- *Extension*: Create occupancy maps and statistics

**Waste Sorting Assistant**

- Classify waste items for proper recycling
- Use waste classification datasets (TrashNet, etc.)
- Guide users in proper waste disposal
- *Extension*: Track recycling statistics

**Wildlife Monitoring**

- Identify and count wildlife species in natural habitats
- Use wildlife camera datasets or create your own
- Implement species recognition and behavior tracking
- *Extension*: Generate biodiversity reports

## Category D: Accessibility/Assistive Technology

**Sign Language Recognition**

- Recognize basic sign language gestures or letters
- Use ASL datasets or create your own gesture collection
- Implement real-time gesture classification
- *Extension*: Translate to text or speech

**Object Recognition for the Visually Impaired**

- Identify and announce common objects in the environment
- Use household object datasets (COCO, Open Images)
- Implement audio feedback system (optional)
- *Extension*: Add spatial relationship descriptions

**Face Expression Recognition**

- Detect emotions from facial expressions
- Use emotion recognition datasets (FER2013, etc.)
- Implement mood tracking or interaction systems
- *Extension*: Generate wellness insights

# Technical Specifications

## Dataset Requirements

**If Using Public Datasets:**

- Proper train/validation/test splits
- Document data source and licensing
- Justify the dataset choice for your application

**If Creating Custom Dataset:**

- Use data augmentation
- Document annotation process and tools used

## Model Requirements

**Performance Targets:**

- Accuracy: ≥70% on test set
- Inference Speed: ≥0.5 FPS on Pi Zero 2W

**Optimization Techniques:**

- Use quantization (INT8) for model compression
- Implement efficient preprocessing pipelines
- Optimize input resolution vs. accuracy trade-offs

## System Requirements

**Real-time Operation:**

- Live camera feed processing
- Visual feedback (bounding boxes, labels, confidence scores)
- Performance monitoring (at least FPS, CPU usage, and temperature - optional)
- Graceful error handling for edge cases

**User Interface:**

- Display predictions with confidence scores
- Show processing statistics
- Include system status indicators
- Provide easy start/stop controls

# Deliverables

## 1. Technical Documentation (40%)

**Project Report:**

- Problem statement and motivation
- Literature review of related work

- Dataset description and preparation process
- Model architecture and training methodology
- Optimization techniques for edge deployment
- Performance evaluation and analysis
- Challenges faced and solutions implemented
- Future improvements and extensions

**Technical Specifications:**

- System architecture diagram
- Hardware setup instructions
- Software dependencies and installation guide
- API documentation (if applicable)

## 2. Source Code and Implementation (10%)

**Code Quality Requirements:**

- Clean, well-documented Python code
- Proper error handling and logging
- Unit tests for critical functions
- Configuration files for easy parameter tuning
- Version control with meaningful commit messages

## 3. Dataset and Model (10%)

**Dataset Package:**

- Organized directory structure
- Annotation files in standard formats (YOLO, COCO, CSV)
- Data splitting information
- Dataset statistics and visualization
- Documentation of collection/annotation process

**Model Package:**

- Trained model in TensorFlow Lite format
- Training history and evaluation metrics
- Model performance benchmarks
- Comparison with baseline models

## 4. Demonstration and Presentation (40%)

**Technical Presentation (suggested: 10-minute video):**

- Problem motivation and approach
- Key technical challenges and solutions
- Performance results and analysis
- Lessons learned and future work

**Live Demonstration (suggested: 5-minute video):**

- Real-time system operation on Pi Zero 2W
- Performance metrics display
- Edge case handling
- User interaction capabilities

# Submission Guidelines

## File Organization

Submit a single ZIP file named `teamX_project_name.zip` containing:

- Complete source code repository
- Trained models and dataset samples
- Technical documentation (PDF)
- Presentation slides (PDF/PowerPoint) (if used)
- Video demonstration (project - Demo)

## Submission Platform

- Upload to the course platform (Moodle)
- One submission per group (ensure all team members are listed as contributors)

## Submission Date

- October 22nd (23:59)